Fiches Infra Big-Data

Spark API

- map(func) : Return a new distributed dataset formed by passing each element of the source through a function func.
- filter(func) : Return a new dataset formed by selecting those elements of the source on which func returns true.
- flatMap(func) : Similar to map, but each input item can be mapped to 0 or more output items (so func should return a Seq rather than a single item).
- mapPartitions(func) : Similar to map, but runs separately on each partition (block) of the RDD, so func must be of type Iterator< T > => Iterator< U > when running on an RDD of type T.
- mapPartitionsWithIndex(func) : Similar to mapPartitions, but also provides func with an integer value representing the index of the partition, so func must be of type (Int, Iterator< T >) => Iterator< U > when running on an RDD of type T.
- sample(withReplacement,fraction, seed) : Sample a fraction of the data, with or without replacement, using a given random number generator seed.
- union(otherDataset) : Return a new dataset that contains the union of the elements in the source dataset and the argument.
- intersection(otherDataset) : Return a new RDD that contains the intersection of elements in the source dataset and the argument.
- distinct([numTasks]) : Return a new dataset that contains the distinct elements of the source dataset.
- groupByKey([numTasks]) : When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable< V >) pairs.
    - Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using reduceByKey or aggregateByKey will yield much better performance.
    - Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional numTasks argument to set a different number of tasks.
- reduceByKey(func,[numTasks]) : When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function func, which must be of type (V,V) => V. Like in groupByKey, the number of reduce tasks is configurable through an optional second argument.
- aggregateByKey(zeroValue, seqOp, combOp, [numTasks]) : When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral “zero” value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in groupByKey, the number of reduce tasks is configurable through an optional second argument.
- sortByKey([ascending],[numTasks]) : When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.
- join(otherDataset, [numTasks]) : When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through leftOuterJoin, rightOuterJoin, and fullOuterJoin.
- cogroup(otherDataset,[numTasks]) : When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable< V >, Iterable< W >)) tuples. This operation is also called groupWith.
- cartesian(otherDataset) : When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
- pipe(command, [envVars]) : Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process’s stdin and lines output to its stdout are returned as an RDD of strings.
- coalesce(numPartitions) : Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.
- repartition(numPartitions) : Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
- repartitionAndSortWithinPartitions(partitioner) : Repartition the RDD according to the given partitioner and, within each resulting partition, sort records by their keys. This is more efficient than calling repartition and then sorting within each partition because it can push the sorting down into the shuffle machinery.
- reduce(func) : Aggregate the elements of the dataset using a function func (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
- collect() : Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
- count() : Return the number of elements in the dataset.
- first() : Return the first element of the dataset (similar to take(1)).
- take(n) : Return an array with the first n elements of the dataset.
- takeSample(withReplacement, num, [seed]) : Return an array with a random sample of num elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
- takeOrdered(n, [ordering]) : Return the first n elements of the RDD using either their natural order or a custom comparator.
- saveAsTextFile(path) : Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call toString on each element to convert it to a line of text in the file.
- saveAsSequenceFile(path) : Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop’s Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).
- saveAsObjectFile(path) : Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using SparkContext.objectFile().
- countByKey() : Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
- foreach(func) : Run a function func on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems.
    - Note: modifying variables other than Accumulators outside of the foreach() may result in undefined behavior. See Understanding closures for more details.

Spark Streaming API

- map(func) : Return a new DStream by passing each element of the source DStream through a function func.
- filter(func) : Return a new DStream by selecting only the records of the source DStream on which func returns true.
- flatMap(func) : Similar to map, but each input item can be mapped to 0 or moreoutput items.
- repartition(numPartitions) : Changes the level of parallelism in this DStream by creating more or fewer partitions.
- union(otherStream) : Return a new DStream that contains the union of the elements in the source DStream and otherDStream.
- count() : Return a new DStream of single-element RDDs by counting the number of elements in each RDD of the source DStream.
- reduce(func) : Return a new DStream of single-element RDDs by aggregating the elements in each RDD of the source DStream using a function func (which takes two arguments and returns one). The function should be associative and commutative so that it can be computed in parallel.
- countByValue() : When called on a DStream of elements of type K, return a new DStream of (K, Long) pairs where the value of each key is its frequency in each RDD of the source DStream.
- reduceByKey(func,[numTasks]) : When called on a DStream of (K, V) pairs, return a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function. Note: By default, this uses Spark’s default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property spark.default.parallelism) to do the grouping. You can pass an optional numTasks argument to set a different number of tasks.
- join(otherStream, [numTasks]) : When called on two DStreams of (K, V) and (K, W) pairs, return a new DStream of (K, (V, W)) pairs with all pairs of elements for each key.
- cogroup(otherStream,[numTasks]) : When called on a DStream of (K, V) and (K, W) pairs, return a new DStream of (K, Seq[V], Seq[W]) tuples.
- transform(func) : Return a new DStream by applying a RDD-to-RDD function to every RDD of the source DStream. This can be used to do arbitrary RDD operations on the DStream.
- updateStateByKey(func) : Return a new “state” DStream where the state for each key is updated by applying the given function on the previous state of the key and the new values for the key. This can be used to maintain arbitrary state data for each key.
- window(windowLength,slideInterval) : Return a new DStream which is computed based on windowed batches of the source DStream.
- countByWindow(windowLength,slideInterval) : Return a sliding window count of elements in the stream.
- reduceByWindow(func,windowLength, slideInterval) : Return a new single-element stream, created by aggregating elements in the stream over a sliding interval using func. The function should be associative and commutative so that it can be computed correctly in parallel.
- reduceByKeyAndWindow(func, windowLength, slideInterval,[numTasks]) : When called on a DStream of (K, V) pairs, returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window. Note: By default, this uses Spark’s default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property spark.default.parallelism) to do the grouping. You can pass an optional numTasks argument to set a different number of tasks.
- countByValueAndWindow(windowLength, slideInterval,[numTasks]) : When called on a DStream of (K, V) pairs, returns a new DStream of (K, Long) pairs where the value of each key is its frequency within a sliding window. Like in reduceByKeyAndWindow, the number of reduce tasks is configurable through an optional argument.
- print() : Prints the first ten elements of every batch of data in a DStream on the driver node running the streaming application. This is useful for development and debugging.
- saveAsTextFiles(prefix, [suffix]) : Save this DStream’s contents as text files. The file name at each batch interval is generated based on prefix and suffix: “prefix-TIME_IN_MS[.suffix]”.
- saveAsObjectFiles(prefix,[suffix]) :Save this DStream’s contents as SequenceFiles of serialized Java objects. The file name at each batch interval is generated based on prefix and suffix: “prefix-TIME_IN_MS[.suffix]”.
- saveAsHadoopFiles(prefix,[suffix]) : Save this DStream’s contents as Hadoop files. The file name at each batch interval is generated based on prefix and suffix: “prefix-TIME_IN_MS[.suffix]”.
- foreachRDD(func) : The most generic output operator that applies a function, func, to each RDD generated from the stream. This function should push the data in each RDD to an external system, such as saving the RDD to files, or writing it over the network to a database. Note that the function func is executed in the driver process running the streaming application, and will usually have RDD actions in it that will force the computation of the streaming RDDs.